

Généralités sur les Design Patterns (Patrons de Conception)

1. Qu'est-ce qu'un design pattern ?

Un **design pattern** est une **solution réutilisable** à un **problème fréquent** rencontré en programmation, surtout en **programmation orientée objet (POO)**.

C'est un **modèle de conception** éprouvé, documenté, et reconnu comme bonne pratique.

Ils ne sont **ni des morceaux de code tout faits**, ni des recettes toutes faites, mais **des structures conceptuelles** applicables à divers langages (comme PHP, Java, C#, etc.).

2. Origine

- Formalisés dans le livre "**Design Patterns: Elements of Reusable Object-Oriented Software**" (1994), par le "**Gang of Four**" (GoF) : *Gamma, Helm, Johnson, Vlissides*.
 - Ils décrivent **23 patrons fondamentaux**, encore largement utilisés aujourd'hui.
-

3. Objectif des design patterns

- Favoriser la **réutilisabilité du code**
 - Faciliter la **maintenance** et **l'évolution**
 - Améliorer la **lisibilité** et **l'architecture**
 - Standardiser** les solutions entre développeurs
-

4. Classification des patterns

A. Créationnels (Creational)

 Gèrent la **création d'objets**

Pattern	Utilité
Singleton	Une seule instance d'une classe
Factory Method	Laisse une sous-classe décider quelle classe instancier
Abstract Factory	Crée des familles d'objets liés
Builder	Construit progressivement un objet complexe
Prototype	Clone des objets existants

B. Structurels (Structural)

➡ Gèrent la **composition des classes et objets**

Pattern	Utilité
Adapter	Convertit l'interface d'une classe
Facade	Simplifie l'accès à un ensemble complexe
Composite	Arborescence d'objets identiques (ex : menus, UI)
Decorator	Ajoute dynamiquement des fonctionnalités
Proxy	Contrôle l'accès à un objet
Bridge	Sépare abstraction et implémentation
Flyweight	Partage des objets communs pour économiser la mémoire

🚦 C. Comportementaux (Behavioral)

➡ Gèrent les **interactions entre objets**

Pattern	Utilité
Observer	Notifie automatiquement les objets dépendants
Strategy	Interchange dynamiquement des algorithmes
State	Change de comportement selon un état
Command	Encapsule une action à exécuter
Iterator	Parcourt une collection sans exposer sa structure
Template Method	Définit un squelette d'algorithme avec des étapes personnalisables
Mediator	Centralise les communications entre objets
Chain of Responsibility	Transmet une requête à travers une chaîne
Visitor	Applique une opération à des objets hétérogènes

5. 🛠 Exemple simple

◆ Singleton en PHP

```
class ConnexionDB {
    private static $instance;

    private function __construct() {}

    public static function getInstance() {
        if (!self::$instance) {
            self::$instance = new ConnexionDB();
        }
    }
}
```

```
    }  
    return self::$instance;  
}  
}
```